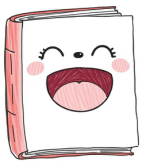
COP 4020 Programming languages
University of Central Florida
C++Assignment: Book and SpellBook Inventory System



In this assignment, you will implement a console-based inventory management system in C++ that:

- Reads book records from a text file.
- Stores them in an Inventory using dynamic allocation and inheritance.
- Uses polymorphism (virtual functions) to:
    - Compute real prices of books.
    - Print book information.
- Provides an interactive menu to:
    - Search for books by name.
    - Print all books.
    - Print the total price of all books.

You must design and implement the classes Book, SpellBook, and Inventory as described below.

Important:

- All instance variables must be **private** all functions will be **public**.
- You must use dynamic binding via **virtual** functions.
- **Follow the output format exactly (spaces, words, and number of decimals).**
- The program must handle missing files gracefully and must not crash.

Input File (bookdata.txt) Each line represents one record. Fields are space-separated. Book names are single words. Assume the given file will be always formatted correctly.

```
b hamlet 158
s necronomicon 500.99 1
b black 189
b othello 15.5
s NinethGate 800 0
```

- **b**: literature book
- **s**: spell book
- **name**: name of the book. A single word.
- **price**: floating-point number
- **canTalk** (spell books only): 1 (true) or 0 (false)

If the file cannot be opened, the program must output

```
Error opening file {filename}
```

and then terminate.

Files to Submit

- main.cpp
- Book.h / Book.cpp
- SpellBook.h / SpellBook.cpp
- Inventory.h / Inventory.cpp

You need to zip the files and submit the zip file. Do NOT put the .cpp/.hpp/.h files in a directory and zip it. **If your submission does not follow this criteria, you will get 0**. No exceptions will be made.

Class Specifications

Book Class

This class will have a **name** (as a string) and a **price** (as a double) instance variable.

- Constructors

- Book()

Parameters: none

Behavior: initializes name = "", price = 100

- Book(std::string btitle, double gprice)

Parameters: book title, base price

Behavior: assigns the given btitle to name instance variable and the given gprice to price.

- Book(std::string line)

Parameters: a record text, e.g. "b hamlet 158"

Behavior: splits on spaces; discards the leading 'b'; uses the next token as name and the third token as price

- Destructor

~Book() must be empty (smart pointers manage memory).

- Getter & Setter Functions

- std::string getName() gets the name value of the Book
- double getPrice() gets the price value of the Book
- void setName(const std::string) sets the name of the Book
- void setPrice(double) sets the price of the Book

- Virtual Functions

- double getRealPrice() // returns price * 0.8
- std::string str() This method's output will be used while printing Book.

Returns: "Literature Book:<name> price:<realPrice>"

Sample output: Literature Book:hamlet price:126.40

There is one space between words. No space after colon (:)

SpellBook Class (inherits from Book)

This class will have a boolean instance variable called canTalk.

- Constructors

- SpellBook()

Behavior: sets canTalk = true

- SpellBook(std::string gname, double gprice, bool cantalkparam)

Parameters: gname, gprice, cantalkparam

Behavior: assigns the given gname to name, gprice to price, and cantalk param to canTalk instance variable respectively.

- SpellBook(std::string line)

Parameters: a record text, e.g. "s necronomicon 500.99 1"

Behavior: splits on spaces; discards 's'; next token is name, then price, then canTalk flag

- Destructor

~SpellBook() must be empty.

- Getter Setter Functions

- bool getCanTalk() returns canTalk instance variable

- Overridden virtual functions

- double getRealPrice()

returns price * 1.1 if canTalk is true, otherwise price

- std::string str()

returns "Spell Book:<name> price:<realPrice>" if canTalk is False. There is only a single space between words; otherwise, "Speaking Book:<name> price:<realPrice>"

Sample outputs:

Speaking Book:necronomicon price:550.09

Spell Book:NinethGate price:800.00

Inventory Class
Access Specifiers & Members
This class will have a vector instance variable, ***vec***, that stores unique pointers to Book instances. Storing pointers will allow us to use inheritance and polymorphism. Storing unique pointers will save us from the burden of memory management.

- Constructor

`Inventory(std::string fname)` calls readData(fname).

- Destructor

`~Inventory()` must be empty.

- Functions

- `void readData(std::string fname)`
opens fname; on failure prints Error opening file {fname} and returns
Otherwise, reads each line, uses line[0] to choose Book or SpellBook, constructs accordingly, then calls add(...)
- `void add(std::unique_ptr<Book> book)`
appends the book object to the vector *vec*
- `bool isEmpty()`
Returns true if vector *vec* is empty
- `std::string str()`
Returns each Book element's str() joined with newlines. You will use polymorphism here to call the overridden str() method.
- `double getRealTotal()`
sums getRealPrice() over vec
- `void searchBook(std::string bname)`
prints matching Book object/s by calling str() or "No result found". If more than one entry is found, each Book is printed on a new line.
- `void menu()`
If `isEmpty()` is true, this function returns immediately (no menu); otherwise, it loops prompting.
Prints ***Enter option search BOOKNAME/all/total/done*** and handles the cases below. Note that the option strings are case-sensitive.
search <name> → calls searchBook(name)
    all → prints str()
    total → prints total from getRealTotal()
    done → exits loop
    any other input → prints "Invalid option" and asks the user to enter the option again.

Program Flow (main.cpp)

- Prompt "Enter filename"
- Read filename, ignore leftover newline.
- Create an Inventory object using the string parameter constructor, passing the file name.
- Call the menu function of the inventory object.

Example run output for the given data (user input is written bold)

Enter filename
bookdata.txt
Enter option search BOOKNAME/all/total/done
search nine
Searched book name: nine
No result found

Enter option search BOOKNAME/all/total/done
search Nin
Searched book name: Nin
Spell Book:NinethGate price:800.000000

Enter option search BOOKNAME/all/total/done
all
The book/s in the inventory is/are
Literature Book:hamlet price:126.400000
Speaking Book:necronomicon price:551.089000
Literature Book:black price:151.200000
Literature Book:othello price:12.400000
Spell Book:NinethGate price:800.000000

Enter option search BOOKNAME/all/total/done
total
Total price of the books is 1641.089000

Enter option search BOOKNAME/all/total/done
Done
Invalid option

Enter option search BOOKNAME/all/total/done
done

Example run output for the given data (use input is written bold)

Enter filename
nonexistent.txt
Error opening file nonexistent.txt

Grading rubrics

- Test Cases: 40 Pts **(Ensure white spaces, and strings in output match as per the example)**
    - File error test (When run with a non-existent file): 5 Pts
    - Search with no result (E.g., on **search nine**): 5 Pts
    - Search with result (E.g., on **search Nin**): 10 Pts
    - All (On **all**, prints every book in file order): 10 Pts
    - Total (On **total**, prints the Total price of the book) : 10 pts
- Implementation in C++: 30 Pts
    - Book class design (10 pts)
    - SpellBook inheritance & overrides (10 pts)
    - Inventory structure & memory management (10 pts)
- File-level header comments (10 pts total)
    - Each file should have a header comment explaining what this code does. Who wrote it? Your NID, name, surname, etc...
- Functional header comments(10 pts)
    - Each function (including constructors, destructors, and helpers) should have a short header/comment explaining its purpose.
        - Purpose of the function (what it does?)
        - Description of parameters and return value.
- Followed Submission Guidelines (10 Pts)
    - All required files are present and correctly named (Book.h/.cpp, SpellBook.h/.cpp, Inventory.h/.cpp, main.cpp, report PDF, etc.)
- The ZIP file must contain all required files directly at the root (no folders inside); if you put everything into a single folder in the ZIP or use any other structure, you receive 0 points for this part.